

Packaging youR code: a beginner's guide to R package development

Jenny Rieck
PHAC/HC R Users Lunch & Learn

29 Jul 2026



Day in the life at HC > ROEB > HPCD

- Health product compliance incident database poorly designed to easily search company or product history
 - No common identifiers
 - Free text entry from public-facing forms
- ATIs, media, and litigation requests regarding compliance files necessitate fast response time
- Developed pipeline of R functions to easily search database and standardize the search results reports for SMEs to review

Before developing a package

- I had put in the effort to create and refine utility functions, BUT:
 - Code was spread across scripts, projects
 - Code wasn't 100% generalizable
 - Multiple versions of the "same" function slightly tweaked
 - Code was poorly documented, lacked robust testing
- Difficulty onboarding new team members to my code
 - How to use the functions? What were the requirements? Do I have the latest version?
- Other solutions (pushing to a git repo, saving to SharePoint) did not solve all the issues

Advantages of packages

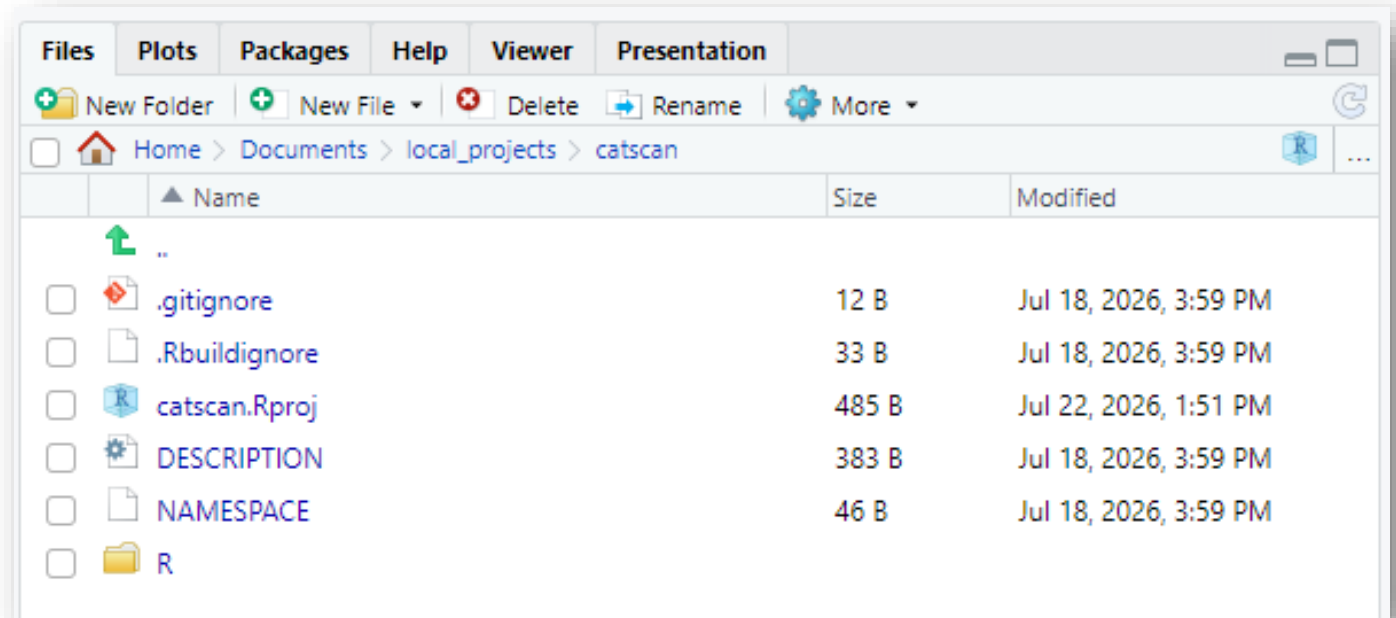
- Really a "package" deal
 - Creates one source of truth
 - Easier as a developer to maintain, manage dependencies, track versions
 - Easier as a user to integrate into a normal R workflow
- "Forces" better code practice
 - Improved reproducibility/generalizability of functions
 - Documentation, documentation, documentation!
- Strongly encourages even better practices
 - Formal unit testing
 - Writing vignettes to serve as tutorials

Getting Started

```
> install.packages("devtools")  
> install.packages("roxygen2")  
> devtools::create_package("mypackage")
```

Alternatively :

- *File > New project > New Directory > R Package*



Bare minimum files

- [DESCRIPTION](#)
 - Meta-data about your package
 - Description, authorship, licensing, version number, dependencies
- NAMESPACE
 - "Generated by roxygen2: do not edit by hand"
- [myproject.Rproj](#)
 - RStudio specific project file
- [./R/](#)
 - Folder for your custom R functions

Function best practices



Do

- Small and focused purpose
- Descriptive, unique, and logical naming conventions
- Write documentation
- Set default argument values
- Validate inputs
- `package::function()` approach
- Use `return()`



Don't

- Rely on global variables (pass through parameters instead)
- Modify global settings:

> `rm(list=ls())`

> `setwd()`

> `source()`

> `Sys.setenv()`

> `options()`

> `Sys.setlocale()`

> `par()`

> `set.seed()`

Examples of an okay vs. better function

```
> library(catscan)  
> fy()  
> gc_fiscal_year()
```


Documenting your functions

- roxygen2 package to help create function documentation
 - > `help("mypackage::myfunction")`
- Add roxygen comments with #' in your function files
 - Code > Insert Roxygen Skeleton

```
1  #' Title
2  #'
3  #' @param date
4  #' @param format
5  #'
6  #' @returns
7  #' @export
8  #'
9  #' @examples
10 #'
```

- `devtools::document()` to generate/update your .Rd help files located in ./man/

Testing your functions

- Ad hoc testing:
 - > `devtools::load_all()`
- Formal testing with `testthat`
 - > `usethis::use_testthat()`
 - Explicitly document a function's expected behavior
 - Automated checks

Vignettes

- Long form, step-by-step tutorials of functions in a package and how they work together
- .Rmd allows you to insert and test code chunks

Build the package!

1. `> devtools::test()`
2. `> devtools::document()`
 - Creates `./man/*.Rd` help files for each function
3. `> devtools::build()`
 - Builds the `package_X.X.X.tar.gz`
4. `> devtools::check()`
5. `> devtools::install(build_vignettes = TRUE)`
6. ****Restart R Session****
7. `> library(mypackage)`

Sharing your package

Updates and versioning

Resources

- R Packages, Hadley Wickham & Jenny Bryan, <https://r-pkgs.org/>

